Microsoft 365 Messaging Support Operations — ITIL Incident & Problem Management Runbook

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary 4

Solution Overview 4

What this is . . . 4

How it works — the messaging support operating model . . . 5

The reference model . . . 6

Benefits . . . 6

Limitations and boundaries . . . 7

Deployment model . . . 7

Requirements 8

R1. A ticketing / ITSM tool with recorded SLAs . . . 8

R2. Messaging admin access for the support tiers (least privilege) . . . 9

R3. Microsoft Graph permission for Service Health (service-availability handling) . . . 9

R4. An on-call rota and escalation contacts . . . 9

R5. A knowledge-base (KB) platform . . . 10

R6. Defined SLA targets and the metric baseline . . . 10

R7. Reporting / analyst workstation . . . 11

Global Requirements — Microsoft 365 / Exchange Messaging Solutions 11

1. Microsoft 365 tenant with Exchange Online . . . 11

2. Administrative access (least-privilege Exchange/365 roles) . . . 12

3. Exchange Online PowerShell (EXO V3) + Graph toolchain . . . 12

4. DNS control for the mail domains . . . 12

5. Hybrid Exchange & Active Directory (for hybrid solutions) . . . 13

6. A pilot scope and change control . . . 13

Implementation Guide 13

Stage 0 — Assess (baseline the current support practice) . . . 14

Stage 1 — Design (define the model) . . . 14

Stage 2 — Configure (build the model in the tools) . . . 17

Stage 3 — Pilot (prove it on a named scope) . . . 18

Stage 4 — Validate (pass/fail gates) . . . 18

Stage 5 — Expand . . . 19

Rollback (process rollback) . . . 19

Testing & Validation 19

T1. Tooling connects and reads (pre-flight) . . . 19

T2. Priority matrix produces consistent triage . . . 20

T3. Escalation criteria are followed (Gate G2) . . . 20

T4. Incident lifecycle completeness . . . 20
T5. Service-availability first step works . . . 20
T6. Problem loop and root-cause (Gate G4) . . . 21
T7. Metrics report is correct and reproducible . . . 21
T8. Mailbox / mail-flow health-check for common tickets . . . 21
T9. KB coverage and currency (Gate G5) . . . 21
T10. Overall validation gate roll-up . . . 22
Troubleshooting 22
Operating-model symptoms . . . 22
Tooling / script symptoms . . . 23
Government cloud reminder . . . 24
Best Practices 24
Operating model and tiers . . . 24
Priority and blast radius . . . 25
Service availability . . . 25
Problem management and root-cause . . . 25
Quality: FCR, CSAT, reopen . . . 26
Knowledge and coaching . . . 26
Automation and safety . . . 26
Deliverables Summary 26
Documentation . . . 26
PowerShell tools (5) — read-only or `-WhatIf` by default, no credentials in code . . . 27
Appendix A — PowerShell Scripts 30

Executive Summary

Outcome. A Microsoft 365 messaging support team that delivers **to SLA with high customer satisfaction** — clean L2–L4 escalation, incidents and problems managed to root cause, service outages triaged fast, and knowledge captured — through **consistent operations rather than firefighting**.

The problem. A messaging support team lives or dies by consistency. When SLAs slip, escalations bounce between tiers, the same mail incidents recur, and customer satisfaction falls, the cost is felt across the whole organization — because *everyone* uses mail. Most teams have the technical skill; what they lack is a **repeatable operating model** for the support practice itself.

The approach. This solution is a **messaging-specific support-operations runbook** built on **ITIL** discipline and scoped tightly to Microsoft 365 messaging (deliberately distinct from general desktop support and from security incident response). It defines:

- **L2–L4 responsibilities and criteria-based escalation** — so issues move to the right tier the first time, always carrying a diagnostic bundle.
- **ITIL incident management** with a **mail-flow blast-radius priority matrix** (one mailbox vs a domain vs the org) — so triage is consistent and SLA-aligned.
- **ITIL problem management** — recurring pain and major incidents drive **root-cause analysis** and permanent fixes, with known-error workarounds published for the front line.
- **On-call and service-availability handling** — a rota plus a disciplined way to read Microsoft's **Service Health Dashboard** and advisories, answering "is it us or Microsoft?" in minutes.
- **Quality and knowledge** — SLA governance, **First-Call-Resolution**, **CSAT**, **reopen-reduction**, and a living **knowledge base + coaching/onboarding** workflow that compounds team quality.

It ships with **five PowerShell tools** (a Service Health triage helper, a mailbox/mail-flow health-check, an SLA/ticket-metric rollup report, a recurring-incident/problem-candidate analyzer, and a KB/runbook config helper) — all **read-only or `-WhatIf` by default**, using modern authentication, with **no credentials in code**.

Why it is low-risk. Adopting the runbook changes **process and ticketing configuration — not production mail flow**. No connector, transport rule, hygiene policy, DNS record, or authentication setting is created or changed. Rollout follows the house **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model: the model is proven on a **named pilot scope** against explicit **pass/fail gates** (SLA attainment, escalation accuracy, reopen rate, a working problem loop, KB coverage) before it reaches the full team, and the **rollback** is a simple revert to the previously documented process. The result is measurable improvement — fewer repeat incidents, cleaner escalation, higher CSAT — at a fraction of the cost of a consulting engagement.

Solution Overview

What this is

This is a **messaging-specific support-operations runbook** for a team that supports Microsoft 365 messaging — Exchange Online and/or Exchange **hybrid** (on-premises Exchange + Active Directory + Azure AD Connect) — at **Level 2, Level 3, and Level 4**. It defines *how the support practice runs*: who owns what across L2–L4, how issues escalate cleanly, how **ITIL incident management** and **problem management** are applied to mail issues, how on-call and service-availability handling work, how the team is measured (SLA, First-Call-Resolution, CSAT, reopen rate), and how the knowledge base and coaching keep quality high. The outcome is a support team that delivers **to SLA with a positive customer experience** through consistent operations rather than firefighting.

Scope boundary — read this first. This runbook is scoped to **messaging support operations**. It is deliberately distinct from, and does **not** replace:

- A **general desktop / end-user-computing support runbook** (device, OS, general Office apps, printing,

identity/password). This runbook covers mail-specific work — mailboxes, mail flow, connectors, hygiene, client mail connectivity, distribution/shared/resource mailboxes, archiving/retention as they surface in tickets.

- **Security incident response** (a confirmed breach, business email compromise as a *security event*,

data-loss investigation, forensics, legal hold for litigation). When a messaging ticket is found to be a **security incident**, this runbook's job is to **recognize it and hand off** to the security incident-response process — it defines the trigger and the handoff, not the security investigation itself.

Where those adjacent processes touch messaging, this runbook names the **handoff**, not the other team's internal steps.

How it works — the messaging support operating model

The operating model has five interlocking parts. Each is defined in the Implementation Guide and named with the house convention `<Purpose> – <Scope> – <Stage>`.

1. **Tiered responsibilities and escalation (L2–L4).** Clear ownership of what each tier does for mail

issues and the exact criteria and path to escalate — so an issue moves up (or across to a vendor or an adjacent team) without bouncing.

1. **ITIL incident management for messaging.** A single, consistent lifecycle for a mail issue —

identify → log → categorize → prioritize → diagnose → resolve → close — with a severity/priority matrix that reflects **mail-flow blast radius** (one mailbox vs a domain vs the whole org).

1. **ITIL problem management for messaging.** When incidents recur or a major incident lands, a

problem record drives **root-cause analysis** and a permanent fix, with a **known-error** entry and workaround captured until the fix ships.

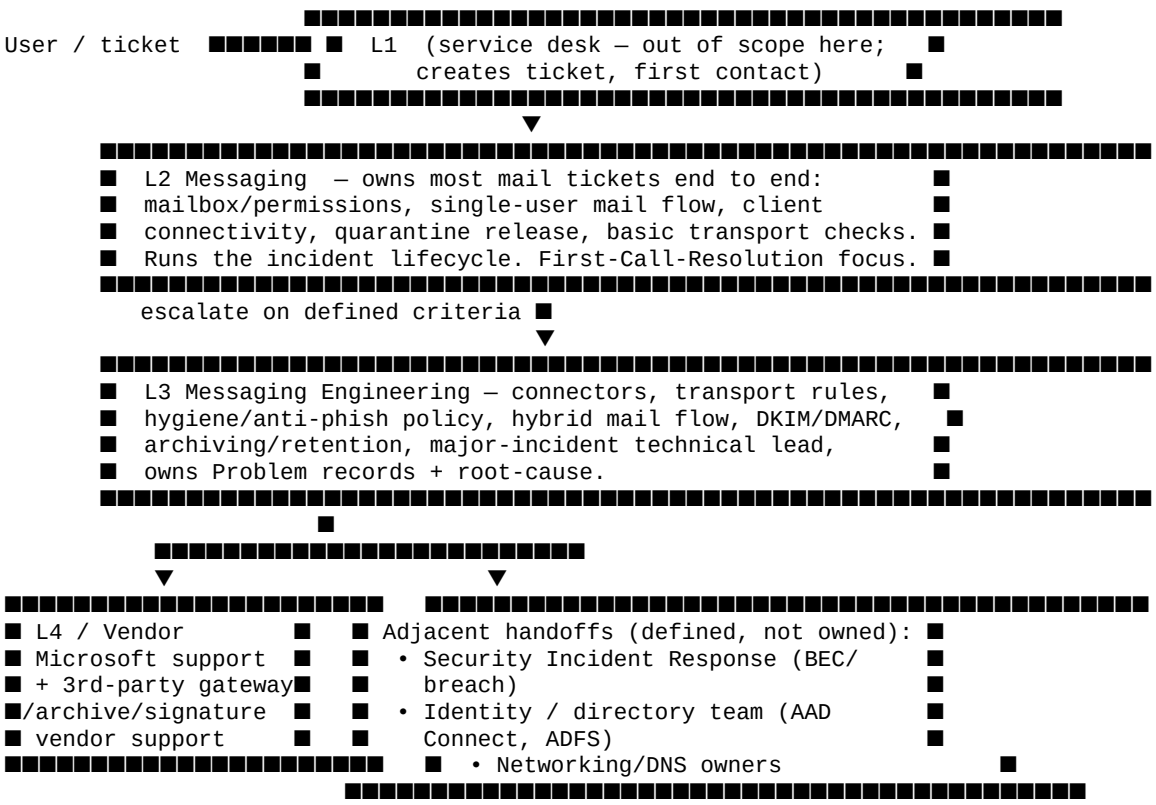
1. **On-call and service-availability handling.** A rota and paging discipline for after-hours major

incidents, plus a defined way to consume Microsoft's **Service Health Dashboard** and **message-center advisories** so the team can tell *"Microsoft-side service issue"* from *"our misconfiguration"* fast.

1. **Quality and knowledge.** SLA governance, **First-Call-Resolution (FCR)**, **CSAT**, and

reopen-reduction practices, backed by a living **knowledge base (KB)** and a **coaching / onboarding** workflow so new analysts ramp and quality compounds.

The reference model



• **L2 (Messaging Support Analyst)** — the front line for mail tickets. Resolves the common, well-documented issues from the KB (mailbox access/permissions, shared/resource mailbox setup, single-user send/receive failures, quarantine/junk questions, client profile issues, distribution group membership). Owns the **incident record** and drives First-Call-Resolution.

• **L3 (Messaging Engineer)** — deep Exchange/EOP/Defender/hybrid work: connectors, transport (mail-flow) rules, anti-spam/anti-phish/anti-malware policy, DKIM/DMARC, accepted domains, hybrid and Edge transport, archiving/retention. Acts as **technical lead on major incidents** and **owns Problem records** and root-cause analysis.

• **L4 (Vendor / Escalation)** — Microsoft support (via the M365 admin center **Support** experience or Premier/Unified) for suspected service-side or product defects, and third-party vendor support for gateway/archive/signature products. L4 is *engaged by* L3, not by users directly.

Benefits

• **Predictable SLA delivery** — a single incident lifecycle and a mail-aware priority matrix mean tickets are handled consistently and to time.

• **Fewer repeat incidents** — problem management converts recurring pain into permanent fixes and

known-error workarounds.

- **Clean escalation** — explicit L2→L3→L4 criteria stop the "hot-potato" bouncing that frustrates users and burns time.
- **Faster "is it us or Microsoft?" calls** — a disciplined way to read the Service Health Dashboard and advisories shortens major-incident triage.
- **Higher CSAT and FCR** — reopen-reduction and knowledge practices raise both the numbers and the felt experience.
- **Faster onboarding** — a KB + coaching workflow ramps new analysts and preserves institutional knowledge.

Limitations and boundaries

- This is an **operating-model and process** solution with supporting automation — it does **not** configure your mail hygiene, connectors, or authentication (those are separate messaging solutions); it defines how the **support team operates** the running service.
- It assumes a **ticketing / ITSM tool** exists (or is stood up per Requirements). It is **tool-agnostic** — it defines the process and metrics, not a specific ITSM product's clicks. ITSM-vendor consoles are deferred to that vendor's own documentation.
- The included PowerShell tools are **read-only or `-WhatIf` by default** and support triage, reporting, and health checks. They are **operational aids**, not a replacement for the ITSM tool or for Microsoft's own portals.
- **Security incident response is out of scope** (see boundary above) — this runbook recognizes and hands off, it does not investigate.
- **Commercial vs Government cloud:** portal URLs and some endpoints differ on **GCC / GCC High / DoD** (e.g. `*.office365.us`, `admin.exchange.microsoft.us`, Graph `graph.microsoft.us`). Where a step or script names a URL/endpoint, government-cloud equivalents are flagged — **verify the correct sovereign-cloud endpoint for your tenant**.

Deployment model

Even though this is a process runbook, it is rolled out with the same house model used for every messaging change, because standing up on-call, escalation, and metrics **touches production support**:

Assess → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback.

- **Assess** — inventory current tickets, tools, tiers, SLAs, and pain (repeat incidents, reopen rate).
- **Design** — define L2–L4 responsibilities, escalation criteria, the priority matrix, on-call rota,

and the metric targets.

- **Configure** — build the runbooks, ticket categories/priorities, KB structure, and the reporting.
- **Pilot** — run the new model on a **named pilot scope** (one queue / one region / one shift) for a defined window.
- **Validate** — measure against the **pass/fail gates** (SLA attainment, escalation-accuracy, reopen rate, CSAT) before expanding.
- **Expand** — roll to the full team; keep improving via problem management and coaching.

The **rollback** for this rollout is documented in the Implementation Guide: because the change is operational (process, categories, rota), rollback means reverting to the previously documented process/queue configuration and pausing the new gates — no production mail-flow object is changed by adopting the runbook itself.

Requirements

These are the **solution-specific** prerequisites for running Microsoft 365 messaging support operations with ITIL discipline. They build **on** the shared category prerequisites in `global-requirements-messaging.md` (a Microsoft 365 tenant with Exchange Online; least-privilege Exchange/365 admin access; the ExchangeOnlineManagement (EXO V3) + Microsoft Graph toolchain; DNS control; hybrid Exchange/AD access where applicable; and a pilot scope + change control). Verify the shared set first, then confirm the items below. Each item lists **Description / Why required / How to verify / How to implement if missing.**

> *Note on access: this is a *support-operations* solution, so the admin access it needs is **least-privilege operational** access for the support tiers (read-heavy for L2, change-capable for L3), plus the ITSM tooling and people/process items below. It does not itself require any new mail-flow configuration.*

R1. A ticketing / ITSM tool with recorded SLAs

- **Description:** A ticketing / IT Service Management system (any vendor — e.g. ServiceNow, Jira Service Management, Freshservice, Zendesk, Halo) that can hold **incident** and **problem** record types, **categories**, **priorities**, **assignment groups** (for L2/L3), and **SLA timers**, and that can **export tickets to CSV** for reporting.
- **Why required:** The incident and problem lifecycles, the escalation paths, and every metric (SLA, FCR, CSAT, reopen rate) live in this tool. Two of the included scripts read a **CSV export** of tickets, so CSV export is a hard requirement.
- **How to verify:** Confirm you can create an Incident and a Problem record, set category/priority, and export a ticket list to CSV with at least these fields (names vary by tool): a ticket ID, created/opened timestamp, resolved/closed timestamp, priority/severity, category, assignment group, reopened flag or reopen count, first-contact-resolved flag, and CSAT score.
- **How to implement if missing:** Stand up (or extend) the ITSM tool; define the record types,

categories, priorities, and assignment groups per the Implementation Guide; enable CSV export.

R2. Messaging admin access for the support tiers (least privilege)

- **Description:** Operational Microsoft 365 access scoped to each tier: **L2** typically needs a

read/triage posture (e.g. **View-Only Organization Management**, plus targeted role groups such as **Help Desk** or **Recipient Management** for the specific tasks it owns — reading mail flow, releasing quarantine, managing mailbox permissions); **L3** needs **change-capable** roles (e.g. **Organization Management** / **Hygiene Management** / **Records Management** as the task requires). Delegated, role-based, MFA-protected.

- **Why required:** L2 must diagnose without the ability to make broad production changes; L3 must be

able to remediate. The included scripts connect with the **operator's own** modern-auth account and inherit its permissions — they carry **no credentials**.

- **How to verify:** In the Microsoft 365 admin center / **Exchange admin center (EAC)** → **Roles**, confirm

the role-group membership for representative L2 and L3 accounts. Confirm `Connect-ExchangeOnline` and, where used, `Connect-MgGraph` succeed for those accounts.

- **How to implement if missing:** Have a Global/Exchange Administrator create/assign the least-privilege

role groups for each tier under change control. Least privilege is version-independent guidance; **verify exact role-group names/capabilities in your tenant** as Microsoft periodically adjusts them.

R3. Microsoft Graph permission for Service Health (service-availability handling)

- **Description:** The ability to read the **Service Health** / **Service Communications** data

programmatically — Microsoft Graph delegated scope `ServiceHealth.Read.All` (used by the Service Health triage helper), *or* equivalent portal access to the **Service Health Dashboard** in the M365 admin center for teams that read it manually.

- **Why required:** Fast "is it us or Microsoft?" triage depends on reading current **service issues**

and **advisories**. `ServiceHealth.Read.All` requires **admin consent** and an appropriate admin role to view service health.

- **How to verify:** Confirm the operator can open **Health** → **Service health** in the M365 admin center;

for the script path, confirm `Connect-MgGraph -Scopes "ServiceHealth.Read.All"` consents and `Get-MgServiceAnnouncementIssue` returns data. **Verify scope availability in your tenant/cloud** — Government clouds use `graph.microsoft.us`.

- **How to implement if missing:** Have an administrator grant/consent the delegated scope, or designate

who has portal Service Health visibility for on-call.

R4. An on-call rota and escalation contacts

• **Description:** A defined **on-call rotation** for after-hours **major incidents** (who is paged, in what order, with a documented secondary/backup), plus current **escalation contacts**: L3 engineering, the Microsoft support entry point (M365 admin center Support / Premier/Unified contract details), and the third-party vendor support contacts (gateway, archive, signature).

• **Why required:** Major messaging incidents (org-wide mail-flow outage, domain-wide auth failure) happen outside business hours; a rota and current contacts are what turn a page into a coordinated response.

• **How to verify:** Confirm a published rota (in the ITSM tool or a paging tool such as PagerDuty/Opsgenie or an on-call calendar) and a current contacts list; confirm the Microsoft support entitlement is known.

• **How to implement if missing:** Build the rota, assign primary/secondary per period, and record escalation contacts in a single, access-controlled runbook location.

R5. A knowledge-base (KB) platform

• **Description:** A platform to hold **KB articles** and **named runbooks** (a wiki, the ITSM tool's KB module, SharePoint, or similar) with **versioning**, **ownership**, and a **review date** per article.

• **Why required:** First-Call-Resolution, reopen-reduction, and onboarding all depend on current, findable knowledge. The problem-management loop publishes **known-error** workarounds here.

• **How to verify:** Confirm analysts can find, create, and version KB articles, and that each article can carry an owner and a next-review date.

• **How to implement if missing:** Stand up (or nominate) the KB platform and seed the structure defined in the Implementation Guide (categories mirror the ticket categories).

R6. Defined SLA targets and the metric baseline

• **Description:** Agreed **SLA targets** per priority (response and resolution), and a **baseline** for the operational metrics you will manage to: SLA attainment, **First-Call-Resolution (FCR)**, **CSAT**, and **reopen rate**.

• **Why required:** You cannot manage to SLA or show improvement without agreed targets and a starting baseline; the Reporting script computes attainment against the targets you set.

• **How to verify:** Confirm documented SLA targets per priority and a first measurement of FCR / CSAT / reopen rate exist (even if rough).

• **How to implement if missing:** Agree SLA targets with the service owner, then run the Reporting script once over a recent ticket export to establish the baseline.

R7. Reporting / analyst workstation

• **Description:** A workstation with **PowerShell 5.1 or 7+**, the **ExchangeOnlineManagement (EXO V3)**

module, and (for the Service Health helper) the **Microsoft.Graph** PowerShell module, able to reach the M365 endpoints for your cloud (Commercial or Government).

• **Why required:** The five included scripts run here.

• **How to verify:** `Get-Module -ListAvailable ExchangeOnlineManagement, Microsoft.Graph`; confirm

`Connect-ExchangeOnline` succeeds.

• **How to implement if missing:** `Install-Module ExchangeOnlineManagement` and `Install-Module

Microsoft.Graph` (current user scope), then confirm modern-auth sign-in.

Global Requirements — Microsoft 365 / Exchange Messaging Solutions

These requirements apply to **every** solution in the **Messaging** category (Exchange Online & Exchange hybrid mail flow, connectors, mail hygiene, email authentication, federation, archiving, client connectivity, and messaging support operations). They are the common foundation each solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Messaging solution).*** *The **Exchange admin center (EAC)**, the **Microsoft 365 admin center**, **Exchange Online PowerShell (EXO V3 module)**, **Microsoft Graph**, and on-premises **Exchange Server / Active Directory** are authored at exact, current click-by-click and cmdlet detail from Microsoft's public documentation. Where a capability, default, or portal path is **version- or plan-dependent** (Exchange Server build/CU, EOP vs Defender for Office 365 plan, Exchange Online Archiving / auto-expanding archive availability, Commercial vs Government cloud), it is **cited or flagged "verify in your tenant/version"** — never invented. **Third-party mail hygiene, archiving, and signature/gateway products** are integrated at the pattern level; their internal consoles are deferred to the vendor's own docs. **Commercial vs Government (GCC/GCC High/DoD)** portal URLs and endpoints differ — where a solution's setup materially differs on a sovereign cloud, flag it (per the house Commercial-vs-Government rule). Automation is delivered as **PowerShell** (Exchange Online PowerShell / Graph) — never with hardcoded credentials or secrets in code.*

1. Microsoft 365 tenant with Exchange Online

• **Description:** A Microsoft 365 tenant with **Exchange Online** and the appropriate licensing for the

capability in scope (e.g. Exchange Online Plan 1/2, EOP, **Defender for Office 365** for advanced hygiene, Exchange Online Archiving / auto-expanding archive where required).

• **Why required:** All mail-flow, hygiene, archiving, and client configuration is performed against

this tenant and depends on the right plan.

• **How to verify:** Sign in to the Microsoft 365 admin center; confirm the Exchange Online subscription and license counts under **Billing > Licenses**.

• **How to implement if missing:** Provision/assign the required licensing before starting.

2. Administrative access (least-privilege Exchange/365 roles)

• **Description:** Administrative access with the **least-privilege role** the solution names — typically **Exchange Administrator** (or specific EAC role groups such as Organization Management, Recipient/Records/Hygiene Management), and **Global Administrator** only where genuinely required (e.g. federation/ADFS, connector auth).

• **Why required:** Mail-flow, connector, hygiene, archiving, and federation changes require the corresponding admin role.

• **How to verify:** Confirm the role assignment in the Microsoft 365 admin center / Entra roles.

• **How to implement if missing:** Have a Global Admin assign the least-privilege role under change control.

3. Exchange Online PowerShell (EXO V3) + Graph toolchain

• **Description:** A workstation with the **ExchangeOnlineManagement (EXO V3)** PowerShell module and, where used, **Microsoft Graph PowerShell**; modern-auth / MFA-capable admin sign-in.

• **Why required:** Configuration, automation, and the provided scripts run through EXO PowerShell / Graph; Basic Auth is retired.

• **How to verify:** `Connect-ExchangeOnline` succeeds with the admin account; `Get-Module ExchangeOnlineManagement`.

• **How to implement if missing:** Install the module (`Install-Module ExchangeOnlineManagement`) and confirm modern-auth sign-in before running the steps.

4. DNS control for the mail domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF (TXT), DKIM (CNAME), DMARC (TXT), Autodiscover, and connector/host records) for the domains in scope.

• **Why required:** Email authentication, mail routing/coexistence, connector validation, and client autodiscover all depend on DNS.

• **How to verify:** Confirm access to the authoritative DNS zone for each mail domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow and authentication changes are high-impact).

5. Hybrid Exchange & Active Directory (for hybrid solutions)

- **Description:** For **hybrid / on-premises** solutions, access to the on-premises **Exchange Server**

organization (correct roles/CU), **Active Directory**, **Azure AD Connect** (directory sync), and the relevant network/firewall paths (e.g. for Edge/transport, hybrid connectors, autodiscover).

- **Why required:** Hybrid mail flow, Edge Transport, connectors, and federation read from and change

the on-premises side.

- **How to verify:** Confirm on-prem Exchange admin access, AD access, and that directory sync is healthy.
- **How to implement if missing:** Obtain access and confirm the on-prem Exchange/AD/AAD-Connect health

before making hybrid changes.

6. A pilot scope and change control

- **Description:** A **pilot group / accepted-domain / test mail flow** to validate on before fleet-wide

rollout, and a **change-control** process for production changes (mail flow, connectors, DNS, authentication, hygiene policy are all high-impact).

- **Why required:** Every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**

model; the pilot proves the design and change control protects production mail flow.

- **How to verify:** Confirm a named pilot scope and an approval/change window are available.
- **How to implement if missing:** Define a pilot scope and agree change control with the messaging/

service owners before making production changes.

*The deployment model across every Messaging solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **PowerShell** (Exchange Online PowerShell / Graph) — never with hardcoded credentials or secrets in code.*

Implementation Guide

This guide stands up the messaging support operating model following the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback. Every artifact — runbook, ticket category, priority level, escalation path, KB structure, report — is **named** with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in Testing, Troubleshooting, and the scripts.

> *Because this rollout is **operational** (process, ticket categories, rota, metrics), it does not change any production mail-flow object. The rollback is therefore a **process rollback** (revert to the previously documented process and pause the new gates), defined at the end of this guide.*

Stage 0 — Assess (baseline the current support practice)

Goal: know where you are before changing anything.

1. Pull a recent **ticket export** (last full month) from the ITSM tool as CSV (per Requirement R1).
2. Run the Reporting script to compute the current baseline:

`.\Get-MessagingSlaMetricsReport.ps1 -TicketCsv .\tickets-lastmonth.csv` — capture SLA attainment, FCR, CSAT, and reopen rate.

1. Run the recurring-incident analyzer to find your top repeat offenders:

`.\Find-RecurringMessagingIncidents.ps1 -TicketCsv .\tickets-lastmonth.csv` — these become the first Problem candidates.

1. Inventory current tiers, tools, SLAs, on-call arrangement, and KB state.

Assess artifacts: `Baseline Metrics – Messaging Support – Assess`, `Repeat-Incident Candidates – Messaging – Assess`.

Stage 1 — Design (define the model)

1.1 Tiered responsibilities — L2 / L3 / L4

Define, in writing, what each tier owns for **messaging** work. Publish as the runbook `Tier Responsibilities – Messaging – Design`.

Tier	Owns (messaging)	Does NOT own
L2 Messaging Support Analyst	Mailbox access & permissions (full-access/send-as/send-on-behalf), shared/resource mailbox setup, single-user send/receive failures, quarantine release & junk questions, client mail profile (Outlook/OWA/mobile) connectivity, distribution-group membership, basic single-user mail-flow checks (message trace for one sender/recipient), out-of-office/forwarding. Owns the **incident record**; drives FCR.	Broad transport-rule / connector / hygiene-policy changes; hybrid config; DKIM/DMARC changes; problem/root-cause ownership.

Tier	Owns (messaging)	Does NOT own
L3 Messaging Engineer	Connectors, transport (mail-flow) rules, anti-spam/anti-phish/anti-malware policy, DKIM/DMARC, accepted domains, hybrid & Edge transport, archiving/retention config, org-wide message-trace analysis. **Technical lead on major incidents. Owns Problem records + root-cause.**	User-by-user password/identity resets (identity team); security-breach investigation (security IR).
L4 Vendor / Escalation	Microsoft support engagement for suspected service-side defects; third-party vendor support (gateway, archive, signature). **Engaged by L3**, not by users.	Day-to-day ticket handling.

1.2 Escalation paths and criteria

Publish `Escalation Matrix – Messaging – Design`. Escalation is **criteria-based**, not effort-based — an analyst escalates when a **defined condition** is met, not merely when "stuck".

- **L2 → L3 when any of:** the fix requires a change to a **connector / transport rule / hygiene policy / accepted domain / DKIM-DMARC / hybrid config; the issue affects** more than one mailbox / a whole domain / the org**; message trace shows a** transport-side **failure L2 cannot resolve; or the agreed** L2 time-box** for the priority is exceeded.
- **L3 → L4 (Microsoft) when:** evidence points to a **service-side issue or product defect** (Service Health shows a related incident, or behavior contradicts documented product behavior and is reproducible), and L3 has collected the diagnostic bundle (message trace, timestamps, message IDs, affected recipients, `Get-` output).
- **L3 → L4 (third-party vendor) when:** the failure is isolated to a gateway/archive/signature product.
- **Cross-hand-off (not an escalation up the mail tier):** if the ticket is a **security incident** (see 1.5), hand off to **Security Incident Response**; if it is an **identity/directory** problem (AAD Connect sync, ADFS), hand to the identity team; if **DNS/network**, to those owners.

Every escalation must carry the **diagnostic bundle** (see the KB template in 3.4) so the receiving tier does not restart triage.

1.3 Priority / severity matrix (mail-flow blast radius)

Publish `Priority Matrix – Messaging – Design`. Priority is driven by **blast radius + business impact**, because a mail issue's severity is defined by *how much mail it stops*.

Priority	Definition (messaging)	Example
P1 / Critical	Org-wide or domain-wide mail flow down, or inbound/outbound mail failing for a large population; major security-relevant mail event in progress.	No mail in or out for the tenant; accepted domain not delivering.
P2 / High	A department, site, or shared business mailbox affected; a connector or transport rule misbehaving for a group; delayed mail queues building.	A partner inbound connector rejecting; a distribution list not delivering.
P3 / Medium	Single-user send/receive problem, mailbox-permission issue, client connectivity for one user.	One user can't send externally; full-access permission not applied.
P4 / Low	Cosmetic / how-to / non-blocking request.	"How do I set an auto-reply?"; signature question.

Attach **response** and **resolution** SLA targets per priority (from Requirement R6) to this matrix.

1.4 On-call and service-availability handling

Publish `On-Call Runbook – Messaging – Design`.

• **Rota:** primary + secondary per period (from R4), paging order, and the *"page on P1, and on P2 that

is trending to P1"* trigger. Business-hours issues follow the normal queue; after-hours **P1/P2** page on-call.

• **Service-availability first step (every suspected outage):** before deep troubleshooting, **check

Microsoft service health**. Two paths:

• Portal: M365 admin center → **Health** → **Service health** (Government: `admin.microsoft.us`), and the

Message center for advisories/changes. **Verify the correct URL for your cloud.**

• Script: `.\Get-ServiceHealthTriage.ps1` (reads the Service Communications API via Graph) to list

active **Exchange Online** service issues and advisories in one place.

• **The rule:** if Service Health shows a **matching active incident**, the ticket is annotated

"Microsoft-side — tracking MS incident <ID>", communications go out, and the team **monitors** rather than chasing a local root cause. If Service Health is **green**, treat it as a local issue and proceed with local diagnosis. This single discipline is the fastest "is it us or Microsoft?" gate.

1.5 Security hand-off trigger (recognize, don't investigate)

Publish `Security Handoff – Messaging – Design`. If a messaging ticket shows any of: reported/suspected **business email compromise** (unexpected mailbox rules, mass external forwarding, mailbox sending spam), credential compromise, confirmed data exfiltration via mail, or an in-progress phishing campaign that is succeeding — **do not investigate as a support incident.** Capture the

immediate facts, apply only the **agreed containment step the security team pre-authorizes** (if any — e.g. do not delete evidence), and **hand off to Security Incident Response** with the record ID. Document the handoff on the incident.

Stage 2 — Configure (build the model in the tools)

2.1 Ticket categories, priorities, and assignment groups (ITSM tool)

In the ITSM tool, create the **categories** (mirror the KB structure in 3.4): *Mailbox & Permissions*, *Mail Flow (single user)*, *Mail Flow (transport/connector)*, *Hygiene / Quarantine / Anti-Phish*, *Client Connectivity*, *Distribution/Shared/Resource*, *Archiving/Retention*, *Service Availability*, *Security Handoff*. Create the **priorities** P1–P4 from the Priority Matrix with their SLA timers. Create **assignment groups** `Messaging-L2` and `Messaging-L3`. Name this configuration record `Ticket Taxonomy – Messaging – Configure`.

> *Use the KB/runbook config helper to generate a **starter taxonomy + KB skeleton** as JSON/Markdown to import or copy into your tools: `.\New-MessagingRunbookConfig.ps1 -OutputPath .\runbook-config`.*

2.2 Incident-management lifecycle

Publish `Incident Lifecycle – Messaging – Configure`, a single flow every mail incident follows: **Identify** → **Log** → **Categorize (2.1)** → **Prioritize (Priority Matrix)** → **Diagnose** → **Resolve** → **Close**, with:

- **Log** — always create the record (even for FCR-at-first-contact) so metrics are complete.
- **Diagnose** — start from the KB article for the category; for suspected outages run the service-availability first step (1.4) **before** local diagnosis.
- **Resolve** — apply the fix at the correct tier; record the resolution and the KB article used.
- **Close** — set closure code, confirm with the user where the SLA requires it, and record CSAT.

2.3 Problem-management lifecycle

Publish `Problem Lifecycle – Messaging – Configure`. A **Problem record** is opened when either: (a) the recurring-incident analyzer or the team flags **repeat incidents** over threshold, or (b) a **major incident (P1/P2)** closes and requires root-cause. The lifecycle: **Detect** → **Log Problem** → **Investigate & Diagnose (root-cause)** → **Raise Known-Error + Workaround** → **Fix (via change control)** → **Verify** → **Close**, with:

- **Root-cause** documented (the "5 whys" or equivalent) — the *permanent* cause, not the symptom.
- **Known-Error record + Workaround** published to the KB immediately so L2 can mitigate recurrences while the fix is pending.
- **Fix** implemented under **change control** (mail-flow/connector/policy changes are high-impact).
- **Verify** the fix stopped the recurrence (re-run the analyzer next period).

2.4 KB and coaching/onboarding workflow

Publish `KB Structure – Messaging – Configure` (categories mirror 2.1; each article carries **owner + next-review date + version**) and `Coaching Onboarding – Messaging – Configure`:

- **KB article template** (per category): Symptom → Preconditions/scope → Diagnostic steps (portal + EXO PowerShell) → Resolution → When to escalate (with the **diagnostic bundle** to attach) → Related Known-Errors.
- **Onboarding path** for a new analyst: shadow → reverse-shadow → solo-with-review over a defined ramp;

a checklist of the core KB articles and the tools (including the five scripts) they must run.

- **Coaching loop:** periodic **ticket-quality reviews** and **reopen reviews** (why did it reopen?),

feeding corrections back into the KB.

2.5 Metrics and reporting

Publish `Metrics Report – Messaging – Configure`. Define the four managed metrics and how each is computed from the ticket export, then schedule `Get-MessagingSlaMetricsReport.ps1` to run against a periodic export:

- **SLA attainment** — % of tickets meeting response and resolution targets per priority.
- **First-Call-Resolution (FCR)** — % resolved at first contact without escalation/reopen.
- **CSAT** — average satisfaction on closed tickets.
- **Reopen rate** — % of closed tickets reopened within the window (a lagging quality signal).

Stage 3 — Pilot (prove it on a named scope)

1. Choose the **pilot scope** (`Support Pilot – <Queue/Region/Shift> – Pilot`) — one queue, one region,

or one shift — per Requirement (global) R6.

1. Run the full model on the pilot scope for a **defined window** (e.g. 2–4 weeks): new categories,

priority matrix, incident + problem lifecycles, on-call trigger, KB template, and reporting.

1. Hold **daily stand-ups** on the pilot queue; open Problem records for any repeat incident.

Stage 4 — Validate (pass/fail gates)

Do not expand until the pilot meets the gates. Record results in `Validation Gates – Messaging – Validate`.

Pass/fail gates (all must pass):

- **G1 — SLA attainment** for the pilot scope ≥ the agreed target (from R6).
- **G2 — Escalation accuracy** ≥ target: sampled escalations met the L2→L3 criteria (no premature or

missed escalations) — the escalation matrix is being followed.

• **G3 — Reopen rate** for the pilot ≤ baseline (from Stage 0) — quality did not regress.

• **G4 — Problem loop working:** at least the top repeat-incident candidate from Stage 0 has a Problem record with a documented root-cause (or a Known-Error + workaround if the fix is pending).

• **G5 — Coverage:** every pilot ticket category has a current KB article; CSAT captured on closed pilot tickets.

If any gate fails, **stay in Pilot**, fix the cause (e.g. tune the priority matrix, clarify an escalation criterion, add missing KB), and re-measure.

Stage 5 — Expand

1. Roll the validated model to the **full messaging support team**.
2. Move the reporting from pilot-scope to team-wide; set the recurring cadence (weekly ops review, monthly metric review).
1. Keep the **continual-improvement loop** running: recurring-incident analysis → Problem records → root-cause fixes → KB updates → coaching. Reopen and repeat-incident trends should decline over time.

Rollback (process rollback)

Because adopting this runbook changes **process and tool configuration**, not production mail flow, the rollback is:

1. **Pause the new gates** and revert the ITSM tool's categories/priorities/assignment groups to the previously documented configuration (keep a pre-change export of the taxonomy so this is a restore, not a rebuild).
1. Revert to the **previously documented** escalation and on-call arrangement.
2. Keep any **Problem records / KB articles** already created (they are additive and harmless).
3. Record the rollback reason as a Problem input so the next attempt improves.

No connector, transport rule, hygiene policy, DNS record, or authentication setting is created or changed by adopting this runbook, so there is nothing to un-configure on the mail platform itself.

Testing & Validation

Validate the operating model before (Pilot) and after (Expand) rollout. Each procedure states the **expected pass/fail** signal. These map to the pass/fail gates in the Implementation Guide (`Validation Gates – Messaging – Validate`). Run the read-only scripts first — they are safe by default.

T1. Tooling connects and reads (pre-flight)

Procedure

1. `Connect-ExchangeOnline` with an L2 and an L3 account.

2. Run `.\Get-ServiceHealthTriage.ps1 -CheckScope` (the scope/health helper) to confirm each account's

effective role/permission posture and module versions.

1. Run `.\Get-ServiceHealthTriage.ps1` after `Connect-MgGraph -Scopes "ServiceHealth.Read.All"`.

Expected — PASS: both accounts connect; the scope helper reports the expected least-privilege posture (L2 read/triage, L3 change-capable); the Service Health helper returns the current Exchange Online service-issue/advisory list (or an explicit "no active issues"). **FAIL:** a connect error, a missing module, or a permission the tier should not have (over-privileged L2).

T2. Priority matrix produces consistent triage

Procedure

1. Take 10 recent, already-closed tickets of mixed impact.

2. Have two analysts independently assign priority using `Priority Matrix – Messaging – Design`.

Expected — PASS: ≥ 8/10 priorities agree between analysts **and** match the blast-radius definition (org/domain = P1, group/site = P2, single user = P3, how-to = P4). **FAIL:** frequent disagreement → the matrix wording needs tightening before pilot.

T3. Escalation criteria are followed (Gate G2)

Procedure

1. Sample escalated tickets from the pilot window.

2. For each, confirm a **defined L2→L3 criterion** (1.2) was actually met and the **diagnostic bundle**

was attached.

Expected — PASS: escalation accuracy ≥ target; no "escalated because stuck" without a criterion; every escalation carries the bundle. **FAIL:** premature/missed escalations, or bundles missing → coach and re-measure.

T4. Incident lifecycle completeness

Procedure

1. Sample closed pilot incidents.

2. Confirm each has: a logged record, a category, a priority, a resolution note with the **KB article

used**, a closure code, and (where required) CSAT captured.

Expected — PASS: all sampled incidents complete on every field. **FAIL:** missing categories, priorities, or closure codes → metrics will be unreliable; fix the lifecycle enforcement.

T5. Service-availability first step works

Procedure

1. During a suspected outage (or a drill), run the on-call **service-availability first step**: portal Service Health + `.\Get-ServiceHealthTriage.ps1`.
1. Confirm the ticket is annotated with the correct call — *"Microsoft-side, MS incident <ID>"* vs *"local issue, proceed"*.

Expected — PASS: the team reaches an "is it us or Microsoft?" determination within the target time and records the MS incident ID when one matches. **FAIL:** the team chased a local root cause while an active MS incident existed (or vice-versa) → reinforce the first-step discipline.

T6. Problem loop and root-cause (Gate G4)

Procedure

1. Run `.\Find-RecurringMessagingIncidents.ps1 -TicketCsv .\tickets.csv`.
2. Confirm the top candidate has a **Problem record** with a documented **root-cause** (or a published **Known-Error + workaround** if the fix is pending).

Expected — PASS: the top repeat-incident candidate is under problem management with root-cause or a known-error workaround published to the KB. **FAIL:** repeat incidents with no Problem record → the loop isn't running.

T7. Metrics report is correct and reproducible

Procedure

1. Run `.\Get-MessagingSlaMetricsReport.ps1 -TicketCsv .\tickets.csv` on a known export.
2. Manually spot-check a handful of tickets against the reported SLA attainment / FCR / reopen rate.

Expected — PASS: the script's SLA attainment, FCR, CSAT, and reopen rate match the manual spot-check; re-running on the same CSV yields identical numbers (deterministic). **FAIL:** mismatch → check the CSV column mapping (the script's `-ColumnMap` / defaults) against your ITSM export.

T8. Mailbox / mail-flow health-check for common tickets

Procedure

1. For a representative "user can't send/receive" ticket, run `.\Test-MailboxMailFlowHealth.ps1 -Identity user@contoso.com` (read-only).

Expected — PASS: the check returns the mailbox's key health facts (mailbox exists/type, primary SMTP, litigation/retention hold state, forwarding, recent message-trace summary for the user) so L2 can diagnose from one place. **FAIL:** the script errors on a valid mailbox → verify permissions/module.

T9. KB coverage and currency (Gate G5)

Procedure

1. For each ticket **category** (2.1), confirm a current KB article exists with an **owner** and a

next-review date not in the past.

Expected — PASS: every category has a current, owned article. **FAIL:** a category with no article or a stale (past-review) article → assign an owner and update before Expand.

T10. Overall validation gate roll-up

Procedure — Consolidate T1–T9 into the gate sheet: G1 (SLA), G2 (escalation accuracy), G3 (reopen ≤ baseline), G4 (problem loop), G5 (KB coverage + CSAT).

Expected — PASS: all five gates pass → proceed to **Expand**. **FAIL:** any gate fails → remain in **Pilot**, remediate, re-measure.

Troubleshooting

Common failure modes of the messaging support operating model and how to resolve them. This is a diagnostic playbook for the *practice* (SLAs, escalation, quality) and for the included tooling — not a mail-flow diagnostic guide (that lives in the per-category KB articles).

Operating-model symptoms

Symptom: SLAs are missed even though the team is busy

• **Likely cause A — priority not driven by blast radius.** Single-user issues are being worked at the

same urgency as domain-wide ones. **Resolve:** enforce `Priority Matrix – Messaging – Design`; P1/P2 are defined by how much mail is affected, and they get worked first.

• **Likely cause B — incidents not all logged.** First-contact fixes go unlogged, so the metric is

computed on a partial, skewed set. **Resolve:** the Incident Lifecycle requires logging *every* contact; audit with Testing T4.

• **Likely cause C — wrong SLA timers in the ITSM tool. Resolve:** confirm the priority-to-SLA

mapping in `Ticket Taxonomy – Messaging – Configure` matches the agreed targets (R6).

Symptom: tickets bounce between tiers ("hot potato")

• **Likely cause — escalation on frustration, not criteria. Resolve:** re-train to

`Escalation Matrix – Messaging – Design`; escalation requires a *defined condition* (a connector/policy/hybrid change, multi-mailbox/domain/org impact, transport-side trace failure, or the L2 time-box). Sample with Testing T3; coach the misses.

• **Secondary cause — no diagnostic bundle.** The receiving tier restarts triage and pushes back.

Resolve: make the bundle (message trace, timestamps, message IDs, recipients, `Get-` output) mandatory on escalation; the KB template enforces it.

Symptom: the same mail incident keeps recurring

• **Likely cause — no problem management.** Incidents are closed individually; the underlying cause is

never fixed. **Resolve:** run `Find-RecurringMessagingIncidents.ps1` each period, open a **Problem record** for the top candidate, drive root-cause, and publish a **Known-Error + workaround** so L2 can mitigate while the permanent fix is under change control. Verify with Testing T6.

Symptom: low CSAT / high reopen rate

- **Likely cause A — "resolved" without confirmation.** Tickets closed before the user's issue actually cleared. **Resolve:** require confirmation on close for the priorities where SLA demands it; run a **reopen review** and feed reasons into the KB.
- **Likely cause B — stale KB.** Analysts follow old portal paths / retired cmdlet guidance and apply wrong fixes. **Resolve:** enforce owner + next-review date per article (Testing T9); refresh mail-surface steps against current Microsoft docs and your tenant/version.

Symptom: the team can't tell "is it us or Microsoft?" quickly

- **Likely cause — service-availability step skipped.** The team dives into local diagnosis while an active Microsoft incident exists (or chases Microsoft while the fault is local). **Resolve:** enforce the `On-Call Runbook – Messaging – Design` first step — check **Service Health** (portal + `Get-ServiceHealthTriage.ps1`) *before* local diagnosis, and annotate the ticket with the MS incident ID when one matches. Drill it (Testing T5).

Symptom: new analysts ramp slowly / quality varies by person

- **Likely cause — no structured onboarding/coaching. Resolve:** run the shadow → reverse-shadow → solo-with-review path in `Coaching Onboarding – Messaging – Configure`, including running the five scripts as part of ramp; coach from real tickets and reopens.

Tooling / script symptoms

`Get-ServiceHealthTriage.ps1` *returns "Not connected to Microsoft Graph"*

- Run `Connect-MgGraph -Scopes "ServiceHealth.Read.All"` first (**Government:** add `-Environment USGov` or `-Environment USGovDoD`). Confirm with `-CheckScope`. The scope requires **admin consent** — if consent is refused, an administrator must grant it (Requirement R3).

`Get-ServiceHealthTriage.ps1` *shows no issues during a real outage*

- Not all local problems are Microsoft-side. A **green** Service Health means treat it as a **local** issue and proceed with local diagnosis — that is the expected, correct behavior, not a script fault.
- Also check the **Message center** advisories (`-IncludeAdvisories`) for change-driven issues that may not appear as a service incident.

`Test-MailboxMailFlowHealth.ps1` *errors on a valid mailbox*

- Confirm `Connect-ExchangeOnline` succeeded and the **account has permission** to read that mailbox

(tier least-privilege — Requirement R2). A `Get-Mailbox not found` on a real mailbox is almost always a permission or wrong-tenant/cloud issue.

- Message trace only covers **recent** history; for older mail use `Start-HistoricalSearch` (noted in the

script output).

`Get-MessagingSlaMetricsReport.ps1` *numbers look wrong or blank*

- Almost always a **column-mapping** issue. The script warns when an expected column is missing — map

your ITSM export's real column names via `-ColumnMap` (e.g. `@{ Priority='urgency'; SlaMet='sla_flag' }`).

- CSAT must be numeric to average; non-numeric CSAT cells are ignored by design. Boolean-ish flags

(SLA-met, FCR, reopened) are matched against the `-*TrueValues` sets — adjust them to your tool's values (`met`/`yes`/`true`/`1`).

`Find-RecurringMessagingIncidents.ps1` *finds nothing*

- Lower `-Threshold` (default 3) to surface smaller clusters, and confirm the **Subject/Category**

columns are mapped for your export. If subjects are highly templated with IDs, the normalizer already strips numbers/emails/GUIDs — verify it's clustering on the right field.

`New-MessagingRunbookConfig.ps1` *— did it change my tenant?*

- No. It writes **only** to the local `-OutputPath` and makes **no** Microsoft 365 connection. Use

`-WhatIf` to preview. Import/copy the generated `taxonomy.json`, `kb-skeleton\*.md`, and `runbooks.md` into your ITSM/KB tools yourself.

Government cloud reminder

If any script or step fails on an endpoint, confirm you are targeting the **right cloud**: Government tenants use `admin.microsoft.us` / `admin.exchange.microsoft.us`, Graph `graph.microsoft.us` (`Connect-MgGraph -Environment USGov/USGovDoD`), and EXO via the correct `-ExchangeEnvironmentName`. Commercial URLs will not resolve for a Government tenant. **Verify the correct endpoints for your tenant.**

Best Practices

Durable practices for running Microsoft 365 messaging support to SLA with high CSAT. These are the habits that keep the operating model healthy after go-live.

Operating model and tiers

- **Escalate on criteria, never on frustration.** An analyst moves a ticket up only when a defined

condition in `Escalation Matrix – Messaging – Design` is met. This is the single biggest lever against "hot-potato" bouncing.

• **Escalate with the diagnostic bundle, always.** No escalation leaves a tier without message trace, timestamps, message IDs, affected recipients, and relevant `Get-` output. The receiving tier should never restart triage.

• **Keep L2 read-heavy, L3 change-capable.** Least-privilege by tier limits blast radius and forces the right hand-off. Review role-group membership periodically — Microsoft adjusts role capabilities over time; **verify in your tenant**.

• **One incident lifecycle for every mail issue.** Even a first-contact fix gets logged, categorized, and prioritized, or your metrics lie to you.

Priority and blast radius

• **Prioritize by how much mail is affected, not by who is shouting.** Org/domain = P1, group/site = P2, single user = P3, how-to = P4. Business impact modifies within a band; it does not override blast radius for genuine mail-flow outages.

• **Watch P2s trending to P1.** A partner connector rejecting today can become an org-wide problem; page on-call when a P2 is trending.

Service availability

• **Check Microsoft Service Health first, every suspected outage.** Portal (**Health** → **Service health**) plus the Service Health triage script. Reading the **Message center** proactively catches *change-driven* incidents before they become tickets.

• **Record the Microsoft incident ID** on affected tickets so post-incident review can separate "our issues" from "service issues", and so trend reporting is honest.

• **Verify your cloud's endpoints.** Government (GCC High/DoD) uses `*.office365.us`, `admin.exchange.microsoft.us`, and `graph.microsoft.us`; do not assume Commercial URLs.

Problem management and root-cause

• **Turn every recurring pain into a Problem record.** Run the recurring-incident analyzer each period; the top candidates get Problem records. Repeat incidents with no Problem record are unmanaged debt.

• **Publish the Known-Error + workaround immediately.** L2 should never have to re-solve a known problem from scratch while the permanent fix is pending.

• **Document the real root cause, not the symptom.** "Restarted / re-ran it" is not a root cause.

• **Fix under change control.** Mail-flow, connector, hygiene, DKIM/DMARC, and hybrid changes are high-impact — pilot the fix and keep a rollback.

Quality: FCR, CSAT, reopen

- **Reopen rate is your truth serum.** A closed ticket that reopens was not resolved. Run a reopen review

and feed the reason back into the KB.

- **Protect First-Call-Resolution with good KB.** Most FCR gains come from findable, current articles for

the common mail tickets.

- **Close with confirmation where the SLA requires it**, and capture CSAT on close — you cannot improve

what you do not measure.

Knowledge and coaching

- **Every KB article has an owner and a next-review date.** Stale mail knowledge (old portal paths,

retired cmdlet guidance) causes wrong fixes. The EXO/EAC surface changes — keep articles current.

- **Onboard by shadow → reverse-shadow → solo-with-review.** New analysts run the five scripts as part of

ramp so they learn the tooling, not just the theory.

- **Coach from real tickets and real reopens**, not abstractions. Ticket-quality reviews close the loop

between metrics and behavior.

Automation and safety

- **Read-only or `-WhatIf` by default.** The included scripts never change anything unless an explicit

`-Commit`/`-Apply` switch is passed; keep it that way for anything you add.

- **No credentials in code, ever.** Connect interactively with `Connect-ExchangeOnline` /

`Connect-MgGraph` and inherit the operator's least-privilege permissions. For unattended reporting, use a managed identity / app registration with the minimum scope — never a stored password.

- **Do not invent cmdlets or portal paths.** When unsure, verify against Microsoft's current docs and

your tenant/version before publishing a KB step.

Deliverables Summary

Everything included in this solution, on one page.

Documentation

Document	What it gives you
Solution Overview	The messaging support operating model: L2–L4 responsibilities + escalation, ITIL incident/problem management for mail, on-call & service-availability handling, SLA/FCR/CSAT, KB/coaching — and the scope boundary vs desktop support and security IR.
Requirements Guide	Solution-specific prerequisites (ITSM tool + CSV export, tiered least-privilege admin access, Service Health read scope, on-call rota, KB platform, SLA targets/baseline, analyst workstation), each with how to verify/implement — on top of the shared messaging requirements.
Implementation Guide	Ordered rollout on the Assess→Design→Configure→Pilot→Validate→Expand model: define L2–L4 responsibilities and escalation, stand up incident + problem management for messaging, wire on-call + service-availability handling, set FCR/CSAT/reopen practices, operate KB + coaching. Every runbook named; pass/fail gates and a process rollback included.
Testing & Validation	T1–T10 procedures with pass/fail signals, mapped to the validation gates.
Troubleshooting	Common operating-model failure modes (missed SLAs, escalation bouncing, repeat incidents, low CSAT/high reopen, "is it us or Microsoft?" confusion, script/permission issues) and their resolutions.
Best Practices	Durable practices for a high-CSAT messaging support practice.
Executive Summary	The outcome, approach, and why the rollout is low-risk — for sponsors.

PowerShell tools (5) — read-only or `-WhatIf` by default, no credentials in code

#	Script	What it does	Safe default
1	**Get-ServiceHealthTriage.ps1**	**Service Health / message-trace triage helper (also the scope/health pre-flight).** Reads the Microsoft Graph **Service Communications API** (`Get-MgServiceAnnouncementIssue` / message-center advisories) to list active **Exchange Online** service issues and advisories, so on-call can quickly answer "is it us or Microsoft?". `-CheckScope` reports the connected operator's effective role posture and required module versions (the pre-flight in Testing T1). Optional single-user message-trace summary via EXO.	Read-only
2	**New-MessagingRunbookConfig.ps1**	**KB / runbook config helper.** Generates a starter **ticket taxonomy** (categories, P1–P4 priorities), a **KB skeleton** (one stub per category, with owner/next-review placeholders), and the named runbook list as JSON/Markdown to import or copy into your ITSM/KB tools.	Writes only to a local `-OutputPath` (no tenant change); `-WhatIf` supported
3	**Test-MailboxMailFlowHealth.ps1**	**Mailbox / mail-flow health-check for common tickets.** For a given mailbox, returns the key facts L2 needs in one place: mailbox type & primary SMTP, forwarding, hold/retention state, quota, and a recent **message-trace** summary for that user.	Read-only
4	**Get-MessagingSlaMetricsReport.ps1**	**SLA / ticket-metric rollup report.** From an exported ticket **CSV**, computes **SLA attainment** (per priority), **First-Call-Resolution**, **CSAT**, and **reopen rate**, with a configurable column map for any ITSM tool.	Read-only (reads a CSV)
5	**Find-RecurringMessagingIncidents.ps1**	**Recurring-incident / problem-candidate analyzer.** From an exported ticket **CSV**, clusters incidents by category/subject to surface the **repeat offenders** that should become **Problem records** for root-cause.	Read-only (reads a CSV)

The **scope/health helper** capability (pre-flight: confirm the operator's effective least-privilege posture per tier and required module versions) is provided by `Get-ServiceHealthTriage.ps1 -CheckScope` — see Testing T1. This solution ships **exactly the five scripts above**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

Find-RecurringMessagingIncidents.ps1

Get-MessagingSlaMetricsReport.ps1

Get-ServiceHealthTriage.ps1

New-MessagingRunbookConfig.ps1

Test-MailboxMailFlowHealth.ps1